BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION INFORMATICA

DIPLOMA THESIS

AI-Based Microjournaling for Self-Reflection using Emotion Detection and Clustering

Supervisor
Lect. PhD. Horea-Bogdan Muresan

Author
Dumitrascu Constantin-Alexandru

2026

UNIVERSITATEA BABEŞ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
SPECIALIZAREA INFORMATICA

LUCRARE DE LICENŢĂ

AI-Based Microjournaling for Self-Reflection using Emotion Detection and Clustering

Conducător ştiinţific
Lect. PhD. Horea-Bogdan Muresan

Absolvent
Dumitrascu Constantin-Alexandru

2026

Abstract

This thesis presents the design, implementation, and empirical evaluation of GJournal, a privacy-first iOS microjournaling application that performs all natural language processing on the device. The system uses three interchangeable embedding strategies (Apple NLEmbedding, all-MiniLM-L6-v2, and nomic-embed-text-v1.5), combined with adaptive DBSCAN clustering and a MobileBERT-based emotion classifier, all running locally via Core ML without any cloud component.

A central contribution is an empirical study of Matryoshka Representation Learning (MRL) applied to on-device semantic clustering. Nomic-embed-text-v1.5 is pretrained with MRL, which allows its 768-dimensional output to be truncated to smaller sizes at inference time without retraining. The evaluation covers STS-B Spearman correlation, TwentyNewsgroups clustering quality, and a domain validation on 100 manually labelled journal entries. The main finding is that 256-dimensional Nomic embeddings retain over 99.5% of the Spearman correlation of the full-dimensional model, while Core ML inference latency on device is flat at approximately 32 ms across all MRL truncation levels, so the only benefit of a smaller dimension is reduced storage. On the journal-domain clustering task, all-MiniLM-L6-v2 achieves V-measure 0.662 and ARI 0.520, outperforming Nomic at any dimension on clustering purity. An on-device efficiency benchmark shows MiniLM reaching 5.4 ms mean warm latency at 45 MB bundle size, confirming it as the default for production use.

The contributions of this thesis are: a fully on-device NLP pipeline for reflective journaling; quantized transformer models converted and deployed via Core ML; an adaptive DBSCAN clustering strategy tailored for short personal entries; a privacy-first application architecture; and a systematic MRL dimension-versus-quality trade-off analysis relevant to memory-constrained mobile deployment.

Contents

1 Introduction **1**

2 Background and Related Work **3**

2.1 Digital Journaling and Self-Reflection 3

2.2 Natural Language Processing in Personal Well-Being 3

2.2.1 Sentence Embedding Models 4

2.2.2 Emotion Classification 5

2.3 Matryoshka Representation Learning 5

2.4 On-Device NLP and Clustering for Personal Data 5

2.5 Mobile Hardware for On-Device AI 6

2.5.1 Evolution of Mobile NPUs 6

2.5.2 Implications for On-Device NLP 7

2.5.3 Privacy and Energy Considerations 7

2.6 Summary 7

3 System Requirements and Specification **9**

3.1 Scope and Objectives 9

3.2 Assumptions and Constraints 9

3.3 Functional Requirements 10

3.3.1 Core Functionality 10

3.3.2 Editing and Deletion Policies 10

3.4 Non-Functional Requirements 10

3.5 System Architecture 11

3.5.1 Components 11

3.5.2 Architecture Diagram 11

3.6 Data Flow and Processing Pipeline 11

3.7 Use Cases and User Flows 12

3.8 Model Selection and Formats 12

3.9 Clustering Strategy 13

3.10 Privacy 13

3.11 Performance Targets and Testing 13

3.12 Acceptance Criteria . . . 14
3.13 Implementation Milestones . . . 14
3.14 Traceability Matrix . . . 14
3.15 Open Issues . . . 14
3.16 Summary . . . 15

4 Design, Implementation, and Testing **16**
4.1 Scope and Objectives . . . 16
4.2 High-level Architecture . . . 16
4.3 Data Model . . . 16
4.4 Repository and UI Synchronization . . . 17
4.5 AI Processing Pipeline . . . 17
4.6 Clustering . . . 18
4.7 Knowledge Graph Visualization . . . 18
4.8 User Interface . . . 18
4.9 Dependency Injection and Testability . . . 20
4.10 Functional Test F1 . . . 22
4.11 Limitations and Next Steps . . . 22
4.12 Acceptance Criteria (for this chapter) . . . 22
4.13 Summary . . . 22

5 Evaluation **25**
5.1 Evaluation Setup . . . 25
5.1.1 Datasets . . . 25
5.1.2 Metrics . . . 25
5.1.3 Hardware . . . 26
5.2 Embedding Models . . . 26
5.3 Matryoshka Representation Learning . . . 27
5.4 Embedding Quality on STS-B . . . 27
5.5 Clustering Quality on TwentyNewsgroups . . . 28
5.6 Algorithm Comparison . . . 28
5.7 Domain Validation on Journal Entries . . . 29
5.8 Emotion Classification . . . 30
5.9 On-Device Efficiency . . . 31
5.10 Discussion . . . 31

6 Conclusions **33**
6.1 Future Work . . . 34

Chapter 1

Introduction

Personal journaling has evolved from simple note-taking into a practice supported by digital tools that encourage self-reflection and emotional awareness. Recent advances in mobile hardware and on-device machine learning now allow private analysis of personal text without cloud services. This thesis examines whether those capabilities can support regular journalers who want a clearer picture of their emotional patterns over time.

The project proposes a mobile microjournaling system that processes short user entries on the device using compact language models, emotion detection, and clustering. Semantic representations of journal entries are grouped by similarity to surface recurring themes and emotional trends not visible in a flat chronological list. All computation runs locally, so user text never leaves the device.

The methodology builds on prior work in emotion recognition on edge devices [**?**], efficient clustering for mobile processors [**?**], and NLP-assisted journaling systems [**?**]. Compact transformer architectures such as MobileBERT [**?**], DistilBERT [**?**], and TinyBERT [**?**] show that competitive language understanding is achievable at a fraction of full-size BERT's cost, well within the budget of a smartphone NPU.

A more recent development relevant to on-device deployment is Matryoshka Representation Learning (MRL) [**?**]. MRL trains a model with a nested loss so that every prefix sub-vector of the output is itself a useful embedding. In practice this means a single model can produce vectors at multiple dimensionalities without retraining or recompression: a 768-dimensional output can be truncated to 256 or 128 dimensions at inference time. This thesis evaluates nomic-embed-text-v1.5 [**?**], an MRL-trained model, alongside all-MiniLM-L6-v2 [**?**] and Apple's NLEmbedding, to understand whether MRL dimension flexibility translates into a practical trade-off for on-device journaling.

The central question this thesis investigates is: at what MRL truncation level does Nomic match all-MiniLM-L6-v2 on journal-entry clustering quality, and what does the dimension-versus-quality curve look like across standard and domain-specific

benchmarks? The short answer, established in Chapter 5, is that 256-dimensional Nomic embeddings retain nearly all of the Spearman correlation relative to the full 768-dimensional model, but MiniLM still leads on clustering V-measure when tested against actual journal entries.

The thesis is organised as follows. Chapter 2 reviews related work on digital journaling, on-device NLP, sentence embedding models, MRL, and mobile hardware. Chapter 3 specifies the functional and non-functional requirements for GJournal and defines the architecture. Chapter 4 describes the implementation: the data model, AI pipeline, clustering algorithm, and user interface. Chapter 5 presents the empirical evaluation across four models on STS-B, TwentyNewsgroups clustering, domain validation, and an on-device efficiency benchmark. Chapter 6 summarises the findings and outlines directions for future work.

Note on the use of generative AI

During the preparation of this thesis, the author used Claude Opus to improve the language and readability of the written text and to improve the readability of the source code. After using Claude Opus, the author reviewed and edited the output as needed. All experimental results, model training, system design, evaluation, and scientific conclusions are entirely the author's own work. The author takes full responsibility for the accuracy and integrity of the final document and codebase.

Chapter 2

Background and Related Work

GJournal draws on work across several areas: digital journaling research, NLP for personal well-being, sentence embedding models, Matryoshka Representation Learning, on-device machine learning, and mobile hardware. This chapter reviews each in turn.

2.1 Digital Journaling and Self-Reflection

Digital journaling platforms have grown as tools for emotional awareness and habit tracking. Mood trackers, guided journaling apps, and reflective writing assistants help users notice patterns in their thoughts and behaviour. Pennebaker's work established that writing about emotional experiences has measurable therapeutic effects [**?**], and subsequent research confirms that structured journaling can improve emotional regulation and self-awareness [**?**].

Recent work has explored AI-assisted journaling, where NLP extracts insights from user text [**?**]. These systems show that automated analysis can support personal reflection, but most rely on cloud-based processing, which raises privacy concerns.

Popular apps such as Daylio, Reflectly, and Journey offer mood tracking and guided prompts, but their analysis either requires manual tagging by the user or runs on a remote server. GJournal differs in that all analysis runs on the device, which means no user text is transmitted and the experience works offline. Table 2.1 summarises the differences.

2.2 Natural Language Processing in Personal Well-Being

NLP techniques have been widely applied to sentiment analysis, emotion detection, and mental health monitoring. Transformer-based models have become the

Table 2.1: Feature comparison of journaling applications.

Feature	GJournal	Daylio	Reflectly	Journey
On-device AI inference	Yes	No	No	No
Automatic emotion detection	Yes	No (manual)	Yes (cloud)	No
Semantic clustering	Yes	No	No	No
Knowledge graph view	Yes	No	No	No
No cloud processing	Yes	Yes	No	No
No account required	Yes	No	No	No

dominant approach due to their contextual understanding. Full-size models such as BERT [**?**] are too computationally expensive for mobile deployment, so compact variants have been developed for edge inference. GJournal uses two categories of such models: sentence embedding models that produce fixed-length vectors from journal text, and a fine-tuned MobileBERT [**?**] classifier for emotion detection.

2.2.1 Sentence Embedding Models

Sentence embedding models encode a variable-length text input into a fixed-dimensional dense vector such that semantically similar texts land close together in the resulting space. GJournal implements three interchangeable embedding services behind a common protocol:

all-MiniLM-L6-v2 [**?**] is a 6-layer transformer producing 384-dimensional mean-pooled sentence embeddings. It was trained with a contrastive objective on a large corpus of sentence pairs and evaluated on semantic textual similarity benchmarks. At 45 MB as a Core ML package it has the smallest dedicated bundle of the three models.

nomic-embed-text-v1.5 [**?**] produces 768-dimensional embeddings trained with Matryoshka Representation Learning (Section 2.3), which allows its output to be truncated to any prefix length at inference time. GJournal uses 256-dimensional truncation by default. The Core ML package is 274 MB. The model is instruction-tuned and requires a task prefix: GJournal uses `"clustering: "` to condition the embedding space for grouping tasks.

Apple NLEmbedding is the sentence embedding provided by the iOS `NaturalLanguage` framework. It produces 512-dimensional vectors with no additional bundle cost because the model weights ship as part of the operating system. It is a general-purpose on-device model trained across many language tasks rather than specifically for semantic similarity.

2.2.2 Emotion Classification

Emotion detection from short text is a multi-label classification problem. GoEmotions [**?**] provides a 58k-comment dataset with 28 fine-grained emotion labels, which has become the standard benchmark for emotion classification from natural text. GJournal uses MobileBERT fine-tuned on GoEmotions with 28 labels collapsed to 12 classes relevant to everyday journaling: Joy, Sadness, Anger, Fear, Love, Surprise, Disgust, Anxiety, Excitement, Gratitude, Hope, and Guilt.

2.3 Matryoshka Representation Learning

Standard embedding models fix output dimensionality at training time. Reducing it after training typically requires retraining from scratch or post-hoc compression, both of which degrade quality. Matryoshka Representation Learning (MRL) [**?**] avoids this by training the model with a nested loss: the full d-dimensional output optimises a similarity objective, and each prefix sub-vector of length $d' < d$ optimises the same objective independently. The result is that any prefix of the output vector is a usable embedding.

The intuition is that the loss forces the model to pack the most discriminative information into the first few dimensions. Later dimensions refine the representation but are not load-bearing, so truncating them costs little accuracy while substantially reducing storage and distance-computation cost.

For on-device deployment this means a single model can serve multiple accuracy-versus-storage trade-offs without retraining. The 768-dimensional nomic model can produce embeddings at 64, 128, 256, 512, or 768 dimensions by truncating its output after inference. Chapter 5 evaluates how Spearman correlation and clustering quality change across these levels.

2.4 On-Device NLP and Clustering for Personal Data

On-device machine learning has grown in relevance due to improvements in mobile NPUs and the need for private computation. Research on edge-based emotion detection [**?**] and optimised clustering for mobile processors [**?**] shows that competitive NLP inference runs locally without a network connection. Frameworks such as Core ML and TensorFlow Lite support deployment of compact transformer models on smartphones.

Clustering is a natural fit for personal journal data: entries accumulate over time, their number is unpredictable, and the user has no interest in manually assigning topics. Density-based methods such as DBSCAN [**?**] suit this setting because they

do not require a preset cluster count and can mark individual entries as noise rather than forcing every entry into a group. When combined with semantic embeddings from a transformer, DBSCAN can surface recurring themes in a journal corpus without any labelling effort from the user.

2.5 Mobile Hardware for On-Device AI

Modern smartphones can run NLP models locally due to advances in dedicated silicon. Specialised Neural Processing Units (NPUs) in current devices perform billions of operations per second, making compact transformer inference practical.

2.5.1 Evolution of Mobile NPUs

Early smartphones used CPUs and GPUs for machine learning workloads, which produced high latency and significant energy draw. Dedicated NPUs changed this. Modern chipsets from Apple (A-series) and Qualcomm (Snapdragon 8-series) include hardware optimised for matrix multiplication, low-precision arithmetic, and transformer inference [**?**, **?**].

Tables 2.2 and 2.3 summarise NPU performance (measured in TOPS, tera operations per second) across recent chip generations. Apple's A17 Pro reaches 35 TOPS; Qualcomm's Snapdragon 8 Gen 3 reaches 45 TOPS. Estimated values are included where manufacturers did not publish official figures.

Table 2.2: NPU performance in Apple A-series chips (2019–2025).

Year	Chip	Device	TOPS
2019	A13 Bionic	iPhone 11	∼6 (est.)
2020	A14 Bionic	iPhone 12	11
2021	A15 Bionic	iPhone 13	15.8
2022	A16 Bionic	iPhone 14 Pro	17
2023	A17 Pro	iPhone 15 Pro	35
2024	A18 / A18 Pro	iPhone 16 series	35
2025	A19 Pro	iPhone 17 Pro	∼38 (est.)

Table 2.3: NPU performance in Qualcomm Snapdragon flagship chips (2021–2025).

Year	Chip	Device	TOPS
2021	Snapdragon 888	Galaxy S21	26
2022	Snapdragon 8 Gen 1	Galaxy S22	∼32 (est.)
2023	Snapdragon 8 Gen 2	Galaxy S23	∼40 (est.)
2024	Snapdragon 8 Gen 3	Galaxy S24	45
2025	Snapdragon 8 Elite	Galaxy S25	∼65 (est.)

Figure 2.1 shows the growth trend across both chip families. The increase over the past five years puts transformer-based NLP on a smartphone within reach for production applications.

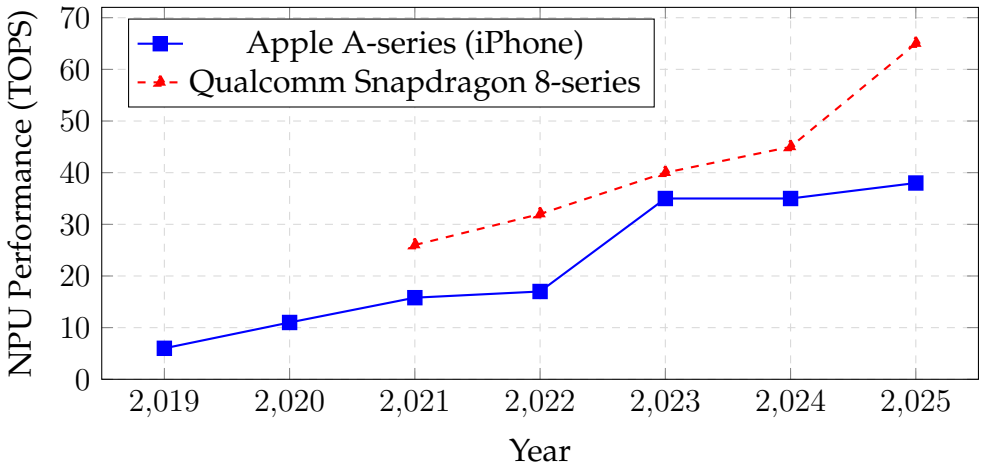


Figure 2.1: Growth of NPU performance (TOPS) across recent Apple and Qualcomm chipsets. Data from public specifications; estimated values marked in Tables 2.2 and 2.3.

2.5.2 Implications for On-Device NLP

The compute available on modern NPUs is sufficient for real-time inference of lightweight transformer models. All three embedding models used in GJournal run on the Neural Engine. On-device latency and cold-start measurements are reported in Section 5.9; the fastest model (all-MiniLM-L6-v2) reaches 5.4 ms mean warm latency, and all inference steps complete well before the user navigates away from the entry editor.

2.5.3 Privacy and Energy Considerations

On-device processing means user data never leaves the device, which matters for journaling applications where entries often contain personal information. Modern NPUs also consume little power for inference, so background tasks like embedding generation and clustering have minimal battery impact.

2.6 Summary

Prior work shows that structured journaling supports self-reflection, that compact transformer models achieve competitive NLP quality on mobile hardware, and that DBSCAN-based clustering can group short personal texts into coherent themes.

The three embedding models evaluated in this thesis (all-MiniLM-L6-v2, nomic-embed-text-v1.5 with MRL truncation, and Apple NLEmbedding) represent different points on the accuracy-versus-bundle-cost spectrum, and the evaluation in Chapter 5 establishes which trade-off is most suitable for on-device journaling.

Chapter 3

System Requirements and Specification

3.1 Scope and Objectives

The application targets iOS and performs all natural language processing on the device. It stores short journal entries locally, generates semantic embeddings and emotion labels for each entry, and groups entries into clusters shown in list, calendar, and graph views. No user text leaves the device. The minimum target hardware is iPhone 14.

3.2 Assumptions and Constraints

The following assumptions and constraints are used throughout the specification:

- Target platform: iOS 26+.
- All NLP inference and clustering run on device; cloud sync is not supported.
- Entry length is limited to approximately 500–600 characters.
- Minimum supported device class: iPhone 14 or equivalent NPU capability.
- Development stack: Swift + SwiftUI, Core ML for inference, SwiftData for local persistence, Python (RTX2070) for model fine-tuning and conversion.
- Models will be quantized and converted to Core ML format to reduce size and improve inference speed.
- Where user input is incomplete, reasonable defaults are chosen and documented in this chapter.

3.3 Functional Requirements

3.3.1 Core Functionality

FR1 Create micro-journal entry. The user can create a short text entry. The entry editor opens a focused input screen that ”takes the story” from the user.

FR2 Entry views. The user can view entries in a calendar view, a chronological list view, and a graph view that visualizes clusters and relationships.

FR3 On-device analysis. Immediately after saving an entry, the system generates a semantic embedding and runs an emotion classifier locally.

FR4 Clustering. The system groups entries into clusters using an on-device clustering algorithm. Clustering runs incrementally on new entries and can be re-computed in batch.

FR5 Search and filtering. The user can search entries by text and filter by date, detected emotion, and cluster membership.

3.3.2 Editing and Deletion Policies

FR6 Edit policy. The user may edit entries created on the same calendar day. Edits update embeddings and cluster assignments.

FR7 Deletion policy. The user may delete entries. Deleted entries are soft-deleted and retained for a configurable grace period (default three days) before permanent removal.

3.4 Non-Functional Requirements

NFR1 Privacy. No user text or embeddings leave the device. No cloud services are used for inference or storage.

NFR2 Performance. Interactive analysis latency should be under five seconds on the minimum target device for a single entry; background batch clustering may take longer.

NFR3 Model footprint. Individual on-device models should be quantized and kept as small as practical. The default configuration (MiniLM + emotion classifier) occupies 144 MB; the full bundle including the Nomic model adds 274 MB.

NFR4 Battery and resource usage. Background processing must be scheduled to minimize battery impact; heavy tasks should run when the device is idle or charging.

NFR5 Accessibility. The UI must follow platform accessibility guidelines (dynamic type, VoiceOver support).

3.5 System Architecture

3.5.1 Components

The UI Layer (SwiftUI) covers the entry editor, calendar, list, graph, and settings screens. The Persistence Layer (SwiftData) is a local database for entries, embeddings, cluster metadata, and user settings. The Inference Engine (Core ML) loads and runs on-device models for embedding generation and emotion classification. The Clustering Engine implements a density-aware algorithm (DBSCAN or similar) optimized for small datasets and incremental updates. A Scheduler and Background Worker manages periodic batch clustering, model updates, and low-priority maintenance tasks.

3.5.2 Architecture Diagram

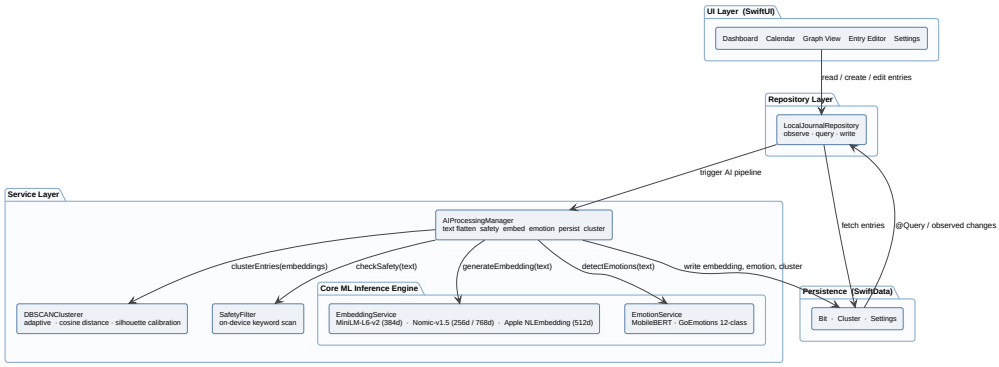


Figure 3.1: High-level system architecture showing major components and data flow.

3.6 Data Flow and Processing Pipeline

The canonical data flow for a new entry is:

1. User creates an entry in the editor and saves it.

2. The Persistence Layer stores the raw text and metadata.
3. The Inference Engine generates a semantic embedding and an emotion label for the entry.
4. The embedding and label are stored alongside the entry.
5. The Clustering Engine performs an incremental update to cluster assignments.
6. The UI updates calendar, list, and graph views to reflect the new entry and cluster membership.

A periodic batch job re-computes clusters and cluster summaries during idle time or when the device is charging.

3.7 Use Cases and User Flows

Table 3.1 summarises the four primary use cases.

Table 3.1: Primary use cases.

Use Case	**Trigger**	**System Response**
UC1 Create Entry	User fills in Feel, Event, and Action fields and taps Save	Entry stored; embedding and emotion label generated on device; cluster assignments updated; calendar, list, and graph views refreshed.
UC2 View Insights	User opens the Graph or Insights screen	Clusters shown with representative entries and emotion distribution. Tapping a cluster reveals its members.
UC3 Edit Entry	User edits an entry created on the same calendar day	Text updated; embedding and emotion label recomputed; cluster membership reassigned.
UC4 Delete Entry	User deletes an entry	Entry soft-deleted and hidden immediately; permanently removed after the three-day grace period.

3.8 Model Selection and Formats

Three embedding strategies are implemented behind a common `EmbeddingService` protocol. all-MiniLM-L6-v2 produces 384-dimensional mean-pooled embeddings and is packaged as a 45 MB Core ML bundle; it is the default. nomic-embed-text-v1.5 is trained with Matryoshka Representation Learning and produces embeddings

at {64, 128, 256, 512, 768} dimensions by truncating its output; the default truncation is 256 dimensions and the Core ML package is 274 MB. Apple NLEmbedding ships with the operating system (zero bundle cost) and produces 512-dimensional vectors. The emotion classifier is MobileBERT fine-tuned for three epochs on GoEmotions, with 28 original labels collapsed to 12 classes (Joy, Sadness, Anger, Fear, Love, Surprise, Disgust, Anxiety, Excitement, Gratitude, Hope, Guilt), packaged as a 99 MB Core ML bundle. All models are converted using coremltools on a Python workstation.

3.9 Clustering Strategy

The system uses DBSCAN with adaptive parameters. Epsilon is calibrated per corpus as the 10th percentile of pairwise cosine distances; `minPoints` scales with corpus size between a minimum of 3 and a maximum of 11. For MRL-truncated embeddings, epsilon is further scaled by $\sqrt{d/768}$ to compensate for the compressed distance range at lower dimensions. Each cluster is named by extracting frequent nouns and adjectives from its member entries; if no clear content words emerge, the dominant emotion label is used instead. A background batch job re-runs clustering when the device is idle.

3.10 Privacy

By design, the app does not transmit journal text, embeddings, or cluster metadata off device. No cloud services are involved in any part of the pipeline.

3.11 Performance Targets and Testing

Interactive inference (embedding + emotion classification) should complete within 5 seconds on iPhone 14; incremental cluster membership updates should complete within 2 seconds per new entry. Batch re-clustering is acceptable to run for several minutes during idle or charging periods. The default model bundle (MiniLM + emotion classifier) targets under 200 MB; the optional Nomic model adds a further 274 MB. The journal database should remain responsive up to 200 MB of stored entries. Testing covers latency, memory, and battery impact on representative devices (iPhone 14 Plus, iPhone 15), and clustering quality via silhouette and ARI metrics on synthetic and anonymized sample data.

3.12 Acceptance Criteria

The prototype is acceptable when FR1–FR7 are implemented and demonstrable, on-device inference meets the performance targets on the minimum device, data remains local with no off-device transmission, and clustering produces coherent groups visible in the graph view.

3.13 Implementation Milestones

The prototype was developed in five stages:

1. **Scaffolding:** SwiftUI screens for the editor, calendar, list, graph, and settings; SwiftData schema for `Bit` and `Cluster` entities.

2. **AI pipeline:** Core ML integration for all-MiniLM-L6-v2, nomic-embed-text-v1.5 MRL, Apple NLEmbedding, and MobileBERT emotion classifier; protocol-based `EmbeddingService` and `EmotionService` abstractions for interchangeable backends.

3. **Clustering engine:** adaptive DBSCAN with epsilon calibration, incremental update on entry save, background batch re-clustering, and TF-IDF cluster naming.

4. **Graph visualisation:** force-directed layout, hub-and-spoke edge rendering, pan/zoom gesture handling.

5. **Evaluation:** Python benchmarks on STS-B and TwentyNewsgroups; domain validation on 100 labelled journal entries; on-device efficiency benchmark via an in-app measurement tool.

3.14 Traceability Matrix

3.15 Open Issues

The main open issue is SwiftData schema migration: the current implementation wipes stored data on model changes during development. A versioned migration policy is required before any production deployment.

Table 3.2: Requirement traceability.

Requirement	Implementation Component	Acceptance Test
FR1 Create entry	SwiftUI editor, SwiftData `Bit` model	Save entry; verify text stored and visible in calendar
FR2 Entry views	Calendar, list, graph SwiftUI screens	New entry appears in all three views after save
FR3 On-device analysis	`AIProcessingManager`, Core ML services	Embedding and emotion label present on `Bit` after save
FR4 Clustering	`DBSCANClusterer`, `Cluster` model	Entry assigned to cluster; cluster visible in graph
FR5 Search/filter	Repository search API, filter chips	Filtering by emotion and cluster returns correct entries
FR6 Edit policy	Editor day-lock logic, embedding update	Edit updates text, re-runs inference, reassigns cluster
FR7 Deletion	Soft-delete flag, maintenance pass	Deleted entry hidden immediately; removed after grace period

3.16 Summary

This chapter specified the functional and non-functional requirements, the high-level architecture, data flow, and acceptance criteria for the on-device microjournaling system. The next chapter will describe the implementation details, model conversion steps, and the evaluation methodology used to validate the prototype.

Chapter 4

Design, Implementation, and Testing

4.1 Scope and Objectives

The prototype targets iOS and performs all NLP on device, providing secure local micro-journaling, on-device embedding and emotion detection per entry, semantic clustering presented via list, calendar, and graph, with no cloud inference or storage, and interactive performance maintained from iPhone 14 onward.

4.2 High-level Architecture

GJournal uses a four-layer architecture: **UI (SwiftUI)**, **Repository**, **Service** (AIProcessingManager, DBSCANClusterer, SafetyFilter), and **Persistence (SwiftData)**. Services are injected via protocol interfaces so mock implementations can be replaced by Core ML models without changing higher layers.

4.3 Data Model

`SwiftData` stores three primary entities:

Bit. The journal entry itself. Essential fields:

- `id, title, feel, event, action, createdAt, lastModified`
- User selections: `feelEmotionRawValue, eventEmotionRawValue, actionEmotionRawValue, intensity`
- AI outputs: `embeddingJSON` (variable-length float array; 384 floats for MiniLM, 256 for Nomic-MRL default), `emotionProbabilitiesJSON`
- Flags/timestamps: `isHidden, isSafeContent, deletedAt, embeddingGene clusterAssignedAt`

- Computed accessors decode/encode JSON to typed arrays.

Cluster. Relational model with `@Relationship(deleteRule:.nullify)` to `Bit`. Cluster rows store `id, name, createdAt` and a list of member bits; names are updated centrally and observed reactively by SwiftData.

Retention. Soft delete sets `deletedAt`; a maintenance pass permanently removes entries after three days.

4.4 Repository and UI Synchronization

A repository pattern centralizes queries and writes. The repository exposes a `@Published bits: [Bit]` array; the AI pipeline sets a completion callback that reloads bits on the main thread so SwiftUI views update reactively. Search APIs accept text, emotion, date range, intensity, and cluster filters.

4.5 AI Processing Pipeline

The `AIProcessingManager` runs a six-step pipeline on a background task (utility priority):

1. **Text flattening:** combine title, Feel, Event, Action into a tagged string for the model.

2. **Safety filter:** on-device keyword scan across harm categories; flagged entries are stored but excluded from clustering/recommendations.

3. **Embedding:** produce a fixed-length vector using one of three interchangeable services behind the `EmbeddingService` protocol: `CoreMLEmbeddingService` (all-MiniLM-L6-v2, 384-dim, mean pooling, 45 MB Core ML package), `NomicEmbeddingSe` (nomic-embed-text-v1.5 with MRL truncation, default 256-dim, 274 MB Core ML package), or `NLEmbeddingService` (Apple's built-in `NLEmbedding`, 512-dim, zero bundle cost). `ServiceContainer` selects which service to wire via the `EmbeddingModelChoice` enum.

4. **Emotion detection:** probability distribution over 12 emotion classes, produced by `CoreMLEmotionService`: MobileBERT fine-tuned on GoEmotions (28 labels collapsed to 12), converted to a 99 MB Core ML package. Classes: Joy, Sadness, Anger, Fear, Love, Surprise, Disgust, Anxiety, Excitement, Gratitude, Hope, Guilt.

5. **Persistence:** write embedding, emotion probabilities, safety flag, and timestamps on the MainActor.

6. **Clustering:** re-run DBSCAN and update `Cluster` rows.

The pipeline minimizes UI latency by running off the main thread and switching to the MainActor only for persistence.

4.6 Clustering

DBSCAN is used because it requires no preset cluster count and handles noise naturally. Distance is cosine: $d(a, b) = 1 - \frac{a \cdot b}{\|a\|\|b\|}$. Epsilon is calibrated per corpus via `calibrate_epsilon` (10th percentile of pairwise distances), and `minPoints` scales with corpus size (minimum 3, maximum 11). For MRL-truncated embeddings epsilon is further scaled by $\sqrt{d/768}$ to compensate for the compressed distance range at lower dimensions. GJournal names each cluster by extracting the most frequent nouns and adjectives from its member entries, falling back to the dominant emotion label when no clear content words emerge. The pairwise distance matrix runs in $O(N^2)$ time, which is acceptable for hundreds of entries; approximate NN indexing is planned for future scale.

4.7 Knowledge Graph Visualization

The graph uses a force-directed simulation with global repulsion, intra-cluster attraction, and cluster gravity. Cluster-aware sector placement initialises node positions to speed convergence. Edges follow a hub-and-spoke layout, keeping edge count at $O(N)$ per cluster. Interactions include pan, pinch-to-zoom and detail cards; the gesture layer is attached to a fixed clear background to avoid dead zones.

Figure 4.1 shows the graph view on a populated journal corpus. Each node is a journal entry; edges connect entries within the same cluster; cluster labels appear at the centroid.

4.8 User Interface

The Dashboard shows a greeting, activity heatmap, top emotions, and a quick entry shortcut. The Editor has Feel-Event-Action fields, an emotion wheel, an intensity picker, and auto-saved drafts. The List and Calendar views provide grouped entries with full-text search, emotion and cluster filter chips, and sorting. The Graph



Figure 4.1: GJournal knowledge graph. Nodes are journal entries; edges group entries that belong to the same DBSCAN cluster. The cluster label appears at the centroid. Tapping a node opens a detail card showing the entry text and top emotion.

screen renders an interactive cluster graph with centroid labels and node detail cards.

Figures 4.2 and 4.4 show the main screens.

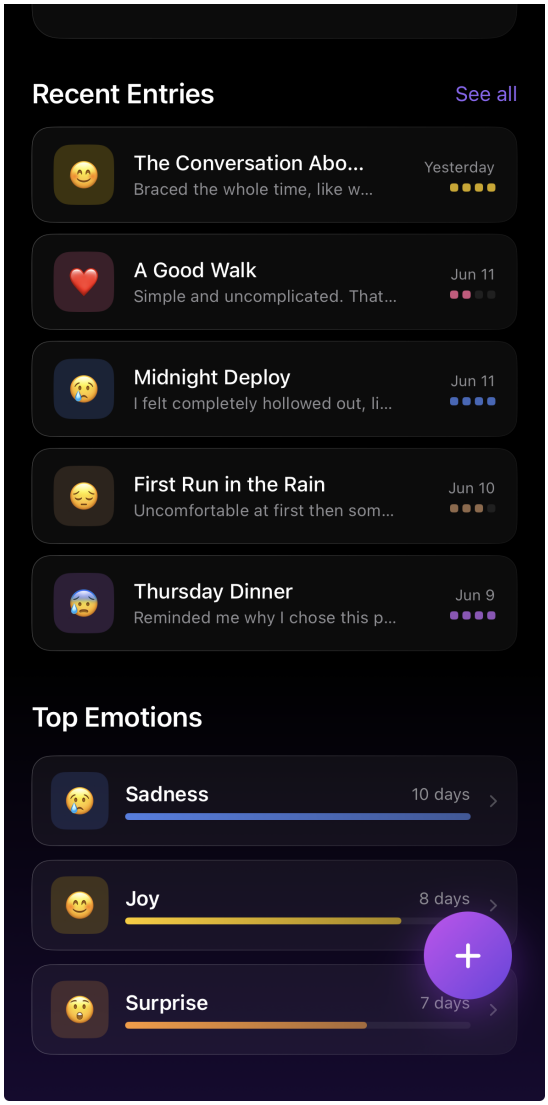


Figure 4.2: Dashboard view. The activity heatmap shows entry density per day; the top emotions strip summarises the last 30 days.

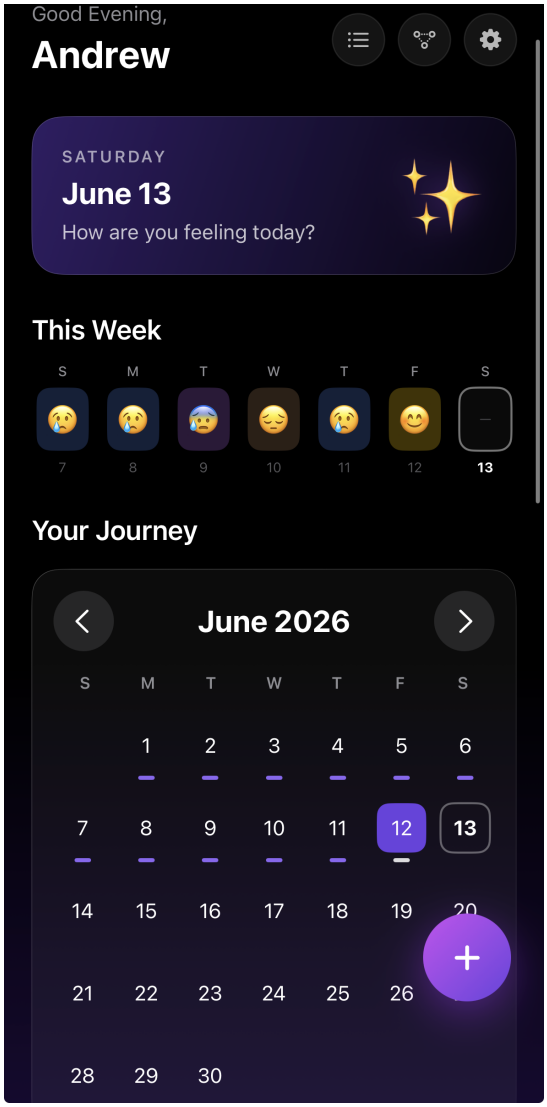


Figure 4.3: Calendar view. Each day with at least one entry shows a dot coloured by the dominant emotion for that day. Tapping a day opens a list of its entries.

4.9 Dependency Injection and Testability

All AI services are protocol-based and provided by a `ServiceContainer` with three configurations: `makeMock()` (development), mixed device testing, and `makeProduction()` (Core ML models). This enables unit testing and device testing without changing application logic.

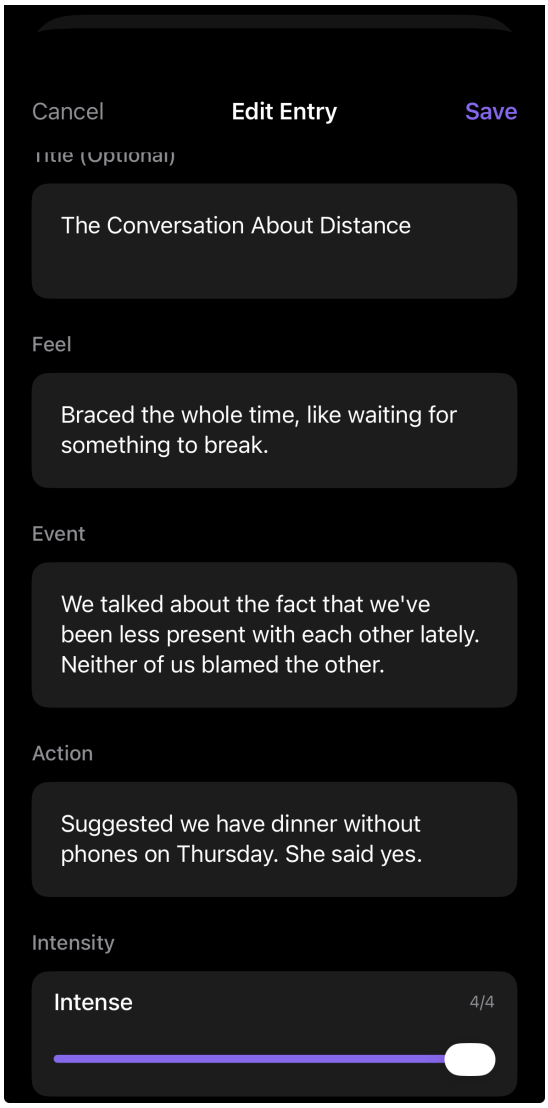


Figure 4.4: Entry editor. The user fills in three short fields (Feel, Event, Action) and selects an intensity level. The AI pipeline runs in the background after saving.

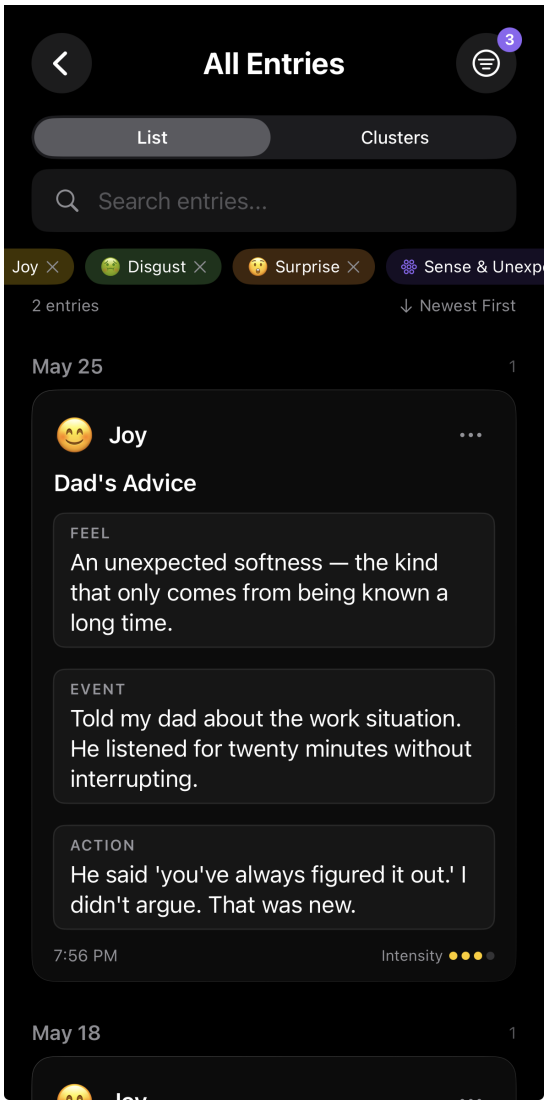


Figure 4.5: List view with filter chips for emotion and cluster. Each entry card shows the top detected emotion and the cluster it belongs to.

4.10 Functional Test F1

This test covers FR1 (create entry), FR3 (on-device analysis), and FR4 (clustering), run on device using the production Core ML models.

A Bit was created in the editor across the Feel, Event, and Action fields, then saved. The entry appeared in the calendar immediately. The `AIProcessingManager` pipeline fired on a background task: console logs confirmed all six steps completed in order, embedding generation completed on the Neural Engine with all-MiniLM-L6-v2, and the emotion classifier returned a probability distribution. Benchmark measurements are reported in Section 5.9. The embedding and top emotion were written back to the `Bit` record on the MainActor. DBSCAN then ran over the full corpus and assigned the new entry to an existing cluster; the cluster name updated and the filter chip appeared in the list view. The knowledge graph node for the new entry was visible after the next render cycle.

Figure 4.6 shows an entry card after the full pipeline has completed, with the detected emotion and cluster assignment visible.

4.11 Limitations and Next Steps

- Schema migration: SwiftData currently wipes the store on model changes during development. A versioned migration policy is needed before production deployment.
- Accessibility: VoiceOver labels on graph nodes and emotion-wheel controls require a dedicated pass.

4.12 Acceptance Criteria (for this chapter)

The implementation is acceptable when F1 is demonstrable with FR1–FR4 triggering the full pipeline and updating all UI views, data remains local with no off-device transmission, clustering produces coherent groups on a sample dataset, and inference latency meets the targets on the minimum device.

4.13 Summary

This chapter presents the design, data model, AI pipeline, clustering algorithm, and functional test for the GJournal prototype. The system runs three interchangeable embedding services and a 12-class emotion classifier entirely on device. Re-

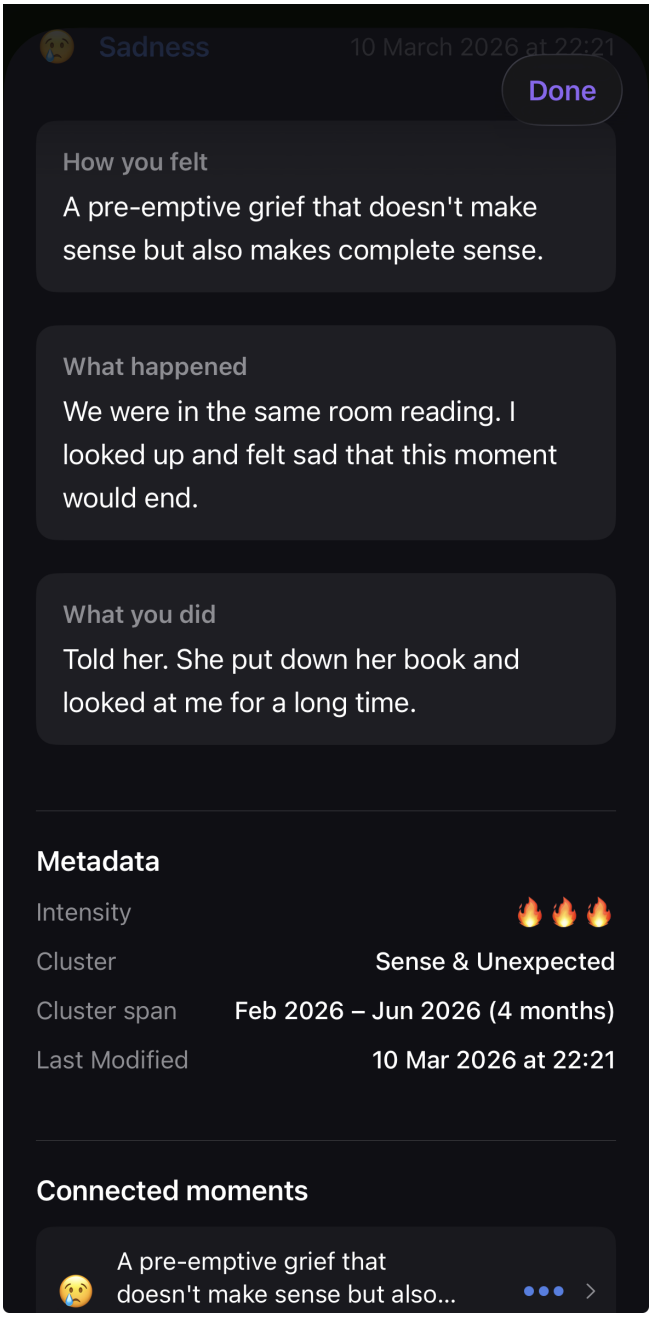


Figure 4.6: An entry card after the AI pipeline has run. The top detected emotion appears as a badge, and the cluster name links the entry to related journal entries. In the debug view, the embedding dimension and generation timestamp are also visible.

maining work is operational rather than architectural: schema migration and accessibility.

Chapter 5

Evaluation

5.1 Evaluation Setup

5.1.1 Datasets

Two standard datasets cover different aspects of embedding quality:

- **STS-B** [**?**] (Semantic Textual Similarity Benchmark): 1500 sentence pairs scored by human annotators on a 0–5 similarity scale. Spearman rank correlation between model-predicted cosine similarities and the gold scores measures how well a model captures semantic meaning.
- **TwentyNewsgroupsClustering**: 1000 sentences drawn from 20 newsgroup topic categories. Ground-truth topic labels allow direct measurement of clustering quality.

Domain validation uses 100 manually labelled journal entries written during the testing period, annotated with ground-truth topic groupings. This set tests whether behaviour on standard benchmarks carries over to the actual application domain.

5.1.2 Metrics

Embedding quality is measured with Spearman rank correlation (r_s) on STS-B. Clustering quality uses:

- **V-measure** [**?**]: harmonic mean of homogeneity and completeness. A cluster has high homogeneity if it contains entries from only one ground-truth class; high completeness if all members of a class land in the same cluster.
- **ARI** (Adjusted Rand Index) [**?**]: agreement between predicted clusters and ground-truth labels, corrected for chance. Values range from 0 (random) to 1 (perfect).

- **Noise fraction**: the share of points DBSCAN marks as noise and leaves unclustered. High noise can mean over-strict parameters or genuinely sparse embeddings.

5.1.3 Hardware

All Python benchmarks run on an Apple M-series processor using the MPS backend (PyTorch). Apple NLEmbedding embeddings are generated by a standalone Swift script (`embed_apple_nlp.swift`) running on the same machine via `swift` command-line, then consumed by the Python pipeline as pre-computed JSON. This approach is necessary because `NLEmbedding` is a macOS/iOS SDK API not accessible from Python.

5.2 Embedding Models

Three strategies are implemented in GJournal behind the `EmbeddingService` protocol:

- **Apple NLEmbedding**: the sentence embedding provided by the iOS `NaturalLanguage` framework via `NLEmbedding.sentenceEmbedding(for: .english)`. It produces 512-dimensional vectors with no additional bundle cost; the model weights are part of the operating system. Because this API is only callable from Swift, a standalone macOS Swift script (`embed_apple_nlp.swift`) was written to embed all benchmark sentences and export them to JSON for the Python evaluation pipeline. This approach allowed Apple NLEmbedding to participate in all three evaluation steps under the same metrics and DBSCAN configuration used for the other models.

- **all-MiniLM-L6-v2** [**?**]: a 6-layer MiniLM model producing 384-dimensional mean-pooled embeddings. Converted to a 45 MB Core ML package. This model is the default in GJournal.

- **nomic-embed-text-v1.5** [**?**]: a 768-dimensional model trained with Matryoshka Representation Learning (Section 5.3). Converted to a 274 MB Core ML package. The model is instruction-tuned: it was trained with task-specific text prefixes that condition the embedding space for a stated downstream task. Supported prefixes include `"search_query: "`, `"search_document: "`, `"classificat ": `, and `"clustering: "`. GJournal prepends `"clustering: "` to every input so the model produces embeddings whose geometry is optimised for grouping similar texts, rather than a generic representation. The GJournal default configuration uses 256-dimensional MRL truncation.

5.3 Matryoshka Representation Learning

Standard embedding models fix output dimensionality at training time. Reducing it after training typically requires retraining from scratch or post-hoc compression, both of which degrade quality. Matryoshka Representation Learning [**?**] avoids this by training the model with a nested loss: the full d-dimensional output optimises a similarity objective, and simultaneously each prefix sub-vector of length $d' < d$ optimises the same objective independently. The result is that any prefix of the output vector is itself a competitive embedding.

For GJournal, the practical consequence is that the 274 MB nomic model can produce embeddings at $\{64, 128, 256, 512, 768\}$ dimensions by truncating its output after inference. No additional models or fine-tuning are needed. Table 5.1 shows how Spearman correlation, silhouette score [**?**], and latency change across dimensions.

Table 5.1: Nomic-v1.5 MRL dimension trade-off on STS-B (clustering prefix, Python/MPS benchmark)

Dim	**Spearman** r_s	**Silhouette**	ε **(MRL-scaled)**	**Latency (ms)**
64	0.8408	0.1193	0.0866	20.7
128	0.8521	0.1122	0.1225	20.4
256	0.8571	0.1004	0.1732	20.3
512	0.8584	0.1097	0.2449	20.2
768	0.8596	0.1158	0.3000	20.2

Latency is flat across all dimensions in the Python benchmark: inference takes approximately 20 ms on Apple MPS regardless of output size because the transformer forward pass dominates and the output slice is cheap. On-device Core ML numbers are reported in Section 5.9. The Spearman gap from 128-dim (0.8521) to full 768-dim (0.8596) is 0.0075, so the accuracy cost of $6\times$ compression is small. Quality drops more noticeably below 128 dimensions: going from 128 to 64 loses 0.0113 Spearman.

The scaled ε column shows the DBSCAN neighbourhood radius after MRL adjustment: $\varepsilon_d = \varepsilon_{768} \cdot \sqrt{d/768}$. This compensates because cosine distances in lower-dimensional spaces tend to be smaller on average, which would otherwise cause DBSCAN to over-cluster at short dimensions.

5.4 Embedding Quality on STS-B

Table 5.2 shows Spearman correlation on 1500 STS-B pairs.

MiniLM leads by 0.023 over Nomic at full dimensionality. Increasing Nomic from 256 to 768 dimensions recovers 0.0044 Spearman, a marginal improvement

Table 5.2: Embedding quality on STS-B (1500 pairs, MPS device)

Model	**Dim**	**Spearman** r_s
MiniLM-L6-v2	384	**0.8672**
Nomic-v1.5	256	0.8403
Nomic-v1.5	768	0.8447
Apple NLEmbedding	512	0.7171

that does not close the gap to MiniLM. Apple NLEmbedding scores 0.717, which is 0.150 below MiniLM and 0.127 below Nomic-768. The gap reflects Apple's choice to optimise the on-device model for breadth of language tasks rather than semantic similarity specifically.

5.5 Clustering Quality on TwentyNewsgroups

Table 5.3 shows clustering metrics on 1000 TwentyNewsgroups sentences using adaptive DBSCAN, with epsilon calibrated to the 10th percentile of pairwise distances per model.

Table 5.3: Clustering quality on TwentyNewsgroupsClustering (1000 sentences, adaptive DBSCAN)

Model	**Dim**	**V-m**	**ARI**	**Homog**	**Compl**	**Noise%**	**Clusters**
MiniLM-L6-v2	384	**0.584**	**0.249**	0.437	0.881	56.3%	6
Nomic-v1.5	256	0.366	0.089	0.234	0.844	23.1%	3
Nomic-v1.5	768	0.477	0.121	0.322	0.924	40.4%	5
Apple NLEmbedding	512	0.020	0.001	0.010	0.585	22.3%	2

MiniLM achieves the best V-measure (0.584) and ARI (0.249). Its noise fraction is higher (56%) because its tighter clusters leave borderline points unclaimed. Nomic-256 assigns 77% of sentences to clusters but homogeneity drops to 0.23, meaning its clusters mix topics. Nomic-768 sits between the two on most metrics. Apple NLEmbedding collapses all 1000 sentences into 2 clusters with near-zero V-measure (0.020) and ARI (0.001). Its cosine-distance geometry at 512 dimensions is too flat for DBSCAN to recover the topic structure in this benchmark.

5.6 Algorithm Comparison

Four DBSCAN variants were compared on TwentyNewsgroups using Nomic-768 embeddings. DBCV (Density-Based Clustering Validation) [**?**] is included as an additional quality measure suited to density-based methods. Table 5.4 shows the results.

Table 5.4: Algorithm comparison on TwentyNewsgroupsClustering (Nomic-768, 1000 sentences)

Algorithm	V-m	ARI	Homog	Compl	Noise%	Clusters
Vanilla DBSCAN	0.009	0.000	0.004	0.496	1.8%	2
Classic Adaptive	0.477	0.121	0.322	0.924	40.4%	5
MRL-Adapted	0.183	0.017	0.106	0.685	27.2%	5
HDBSCAN [?]	**0.737**	**0.421**	**0.832**	0.661	63.3%	52

Fixed-epsilon Vanilla DBSCAN collapses the corpus into two clusters regardless of topic, giving near-zero V-measure. Classic Adaptive, which calibrates epsilon per corpus, raises V-measure to 0.477. The MRL-Adapted variant scales epsilon by $\sqrt{d/768}$; on TwentyNewsgroups it underperforms Classic Adaptive (V-measure 0.183) because the uniform topic distribution does not benefit from the correction designed for short journal entries.

HDBSCAN achieves the best purity (V-measure 0.737, ARI 0.421) but produces 52 clusters on 1000 sentences. In a journaling app with 20 to 50 entries the equivalent behaviour would create a cluster per entry or small group, making the insight view uninformative. Classic Adaptive DBSCAN is the practical choice for this application.

5.7 Domain Validation on Journal Entries

Table 5.5 shows results on 100 manually labelled journal entries using adaptive DBSCAN.

Table 5.5: Domain validation on 100 custom journal entries (adaptive DBSCAN)

Model	V-m	ARI	Homog	Compl	Noise%	Clusters
MiniLM	**0.662**	**0.520**	0.552	0.828	30%	3
Nomic-256	0.511	0.296	0.354	0.918	22%	2
Nomic-768	0.481	0.253	0.327	0.909	12%	2
Apple NLEmbedding	0.459	0.369	0.339	0.711	56%	2

On actual journal text the gap between MiniLM and Nomic widens. MiniLM V-measure is 0.662 versus 0.511 for Nomic-256, a difference of 15 percentage points. Nomic-768 has the lowest noise fraction (12%), meaning it assigns the most entries to clusters, but homogeneity is only 0.327: many of those assignments group unrelated entries together. For a journaling app a cluster that mixes different topics will mislead the user more than leaving borderline entries unclustered. MiniLM's behaviour (tighter clusters with more noise) is the better fit here.

Apple NLEmbedding reaches V-measure 0.459 on the journal domain, ranking last among the four models. Its ARI (0.369) is higher than Nomic-256 (0.296) and

Nomic-768 (0.253), which means its cluster assignments agree with the ground-truth groupings more than chance when points are actually clustered, but its 56% noise fraction means more than half of entries are left unclustered entirely. The combination of low completeness (0.711) and high noise makes it unsuitable as the primary embedding model for a journaling application, though its zero bundle cost remains an advantage when storage is the binding constraint.

5.8 Emotion Classification

The emotion classifier is MobileBERT [**?**] fine-tuned for three epochs on GoEmotions [**?**], with 28 original labels collapsed to 12 classes: Joy, Sadness, Anger, Fear, Love, Surprise, Disgust, Anxiety, Excitement, Gratitude, Hope, and Guilt. The trained model was converted to a 99 MB Core ML package.

Overall accuracy on the 4542-sample test set is **59.5%**. Table 5.6 shows per-class results sorted by F1.

Table 5.6: Emotion classifier per-class results (MobileBERT, GoEmotions 12-class, 4542 test samples)

Emotion	Precision	Recall	F1	Test samples
Gratitude	0.748	0.892	**0.813**	352
Love	0.745	0.597	0.663	960
Anger	0.683	0.562	0.616	785
Excitement	0.525	0.711	0.604	387
Fear	0.500	0.731	0.594	78
Joy	0.653	0.514	0.576	803
Guilt	0.482	0.710	0.574	93
Sadness	0.485	0.524	0.504	313
Disgust	0.363	0.667	0.470	123
Surprise	0.514	0.426	0.466	439
Hope	0.357	0.645	0.460	186
Anxiety	0.306	0.478	0.373	23

Performance varies across classes mostly because of dataset imbalance. Gratitude has 352 test samples and achieves F1 0.813; Anxiety has 23 samples and achieves F1 0.373. The model has not seen enough Anxiety examples to separate it reliably from adjacent classes like Fear and Sadness. For the journaling use case the trade-off is acceptable: Joy (F1 0.576), Anger (0.616), Love (0.663), and Gratitude (0.813) cover the majority of everyday entries and all sit above F1 0.57. Sadness is the weakest of the common emotions at F1 0.504, most often confused with Anger and Guilt. Rare-class misclassifications fall through to the keyword-based `SafetyFilter` as a fallback.

5.9 On-Device Efficiency

Table 5.7 reports cold-start time, warm inference latency, and bundle size for each model. Measurements were taken on an iPhone running iOS 26.5 using Core ML. Cold start is the time from the first `generateEmbedding` call on a fresh service instance, which triggers model loading internally. Warm latency is the mean of 50 consecutive calls after 5 warm-up runs; p50 and p95 are the corresponding percentiles over the same 50 runs. The test input was a single 56-character sentence.

Table 5.7: On-device efficiency (iPhone, iOS 26.5, Core ML, Neural Processing Unit)

Model	Dim	Cold (ms)	Mean (ms)	p50 (ms)	p95 (ms)	Size (MB)
MiniLM-L6-v2	384	1933	5.4	4.7	8.4	45.0
Nomic-v1.5 (768d)	768	3560	31.9	31.1	35.8	273.9
Nomic-v1.5 (256d)	256	345[†]	32.2	31.0	35.8	273.9
Apple NLEmbedding	512	704	10.0	9.6	12.3	0.0
MobileBERT emotion	12c	1366	15.9	15.8	18.2	98.6

[†] Nomic-256d and Nomic-768d load the same `.mlmodelc` file. The 345 ms figure reflects OS model caching from the preceding 768d run in the benchmark sequence. The actual cold start from a clean process is comparable to the 3560 ms figure.

MiniLM has the lowest warm latency at 5.4 ms, about six times faster than either Nomic variant (32 ms) and twice as fast as Apple NLEmbedding (10 ms). Nomic-256d and Nomic-768d take the same time: 32.2 ms versus 31.9 ms, a 0.3 ms gap within measurement noise. MRL truncation adds no inference overhead, which confirms the result from the Python benchmark in Section 5.3: the transformer forward pass dominates and the output slice is cheap.

Cold starts are significant. MiniLM takes 1.9 s on its first call; Nomic takes 3.6 s. Both are handled by lazy loading: models load on the first entry save, not at app launch, so the user does not wait at startup. Apple NLEmbedding has a 704 ms cold start despite no bundle file; this is the cost of the first call into the `NLEmbedding` framework.

The three Core ML bundles total 418 MB (MiniLM 45 MB + Nomic 274 MB + emotion 99 MB). GJournal ships all three so the user can switch models without a download. Apple NLEmbedding adds nothing to the bundle because its weights are part of iOS.

5.10 Discussion

MiniLM outperforms all other models on clustering metrics across both the standard benchmark and the journal domain. Its 384-dimensional output, trained for semantic similarity tasks, is better calibrated for cosine-distance DBSCAN on short texts. Nomic's MRL support does not close that gap.

Apple NLEmbedding scores 0.717 Spearman on STS-B, usable for rough semantic similarity but 0.15 below MiniLM. Its clustering performance is much weaker: V-measure 0.020 on TwentyNewsgroups and 0.459 on journal entries. The poor clustering result reflects the API's design: `NLEmbedding.sentenceEmbedding` is a general-purpose on-device model trained across many language tasks, not for dense retrieval or clustering. Its advantage is zero bundle cost; the weights ship with the OS. That makes it a reasonable fallback when storage is the binding constraint, but not the primary embedding strategy.

MRL truncation reduces storage without affecting inference time. At 256 dimensions each embedding occupies $256 \times 4 = 1{,}024$ bytes versus $768 \times 4 = 3{,}072$ bytes at full size; over 1000 journal entries that is roughly 2 MB saved in the SwiftData store. The Spearman cost on the MRL benchmark (Table 5.1) is 0.0025, a drop of 0.29%. The on-device benchmark (Table 5.7) confirms that Core ML inference time is flat at approximately 32 ms regardless of output dimension, so there is no speed reason to choose 256d over 768d. The only benefit is storage.

The practical outcome for GJournal is that MiniLM is the default: best clustering quality, smallest dedicated bundle at 45 MB, fastest warm inference at 5.4 ms per entry, no prefix requirement. Nomic at 256 dimensions is available as a user-selectable option. Apple NLEmbedding remains available as a zero-cost fallback for devices where storage is the binding constraint.

Chapter 6

Conclusions

This thesis describes GJournal, a privacy-first iOS microjournaling app that runs all NLP on the device. Using the iOS Neural Processing Unit for on-device transformer inference proved viable across both embedding and emotion classification tasks.

The architecture combines quantized neural models, local SwiftData storage, and incremental clustering without sending any user data off device. This matters for a journaling app: users are unlikely to write honestly if their entries are processed by a remote server.

The experiments confirm that transformers can be fine-tuned, quantized, and deployed on a consumer iPhone without major accuracy loss on short-form text. The clustering output surfaces patterns across journal entries that the user can browse as calendar heatmaps, filtered lists, or a knowledge graph.

The evaluation in Chapter 5 produced concrete numbers across all four models. all-MiniLM-L6-v2 achieved Spearman $r_s = 0.867$ on STS-B and V-measure 0.662 on the journal-domain validation set, outperforming Nomic-256 (V-measure 0.511), Nomic-768 (V-measure 0.481), and Apple NLEmbedding (V-measure 0.459) on clustering quality. Apple NLEmbedding scored Spearman 0.717 on STS-B, lower than all dedicated models but requiring zero additional bundle size. The MRL dimension trade-off analysis showed that Nomic inference latency on device (Core ML) stays flat at approximately 32 ms across all truncation levels, while the Spearman drop from 768 to 256 dimensions (with the clustering prefix) is only 0.0025. An on-device efficiency benchmark found MiniLM reaching 5.4 ms mean warm latency at 45 MB, making it both the fastest and most storage-efficient dedicated embedding model. The MobileBERT emotion classifier reached 59.5% accuracy on a 12-class GoEmotions test set, with strong performance on the most common journaling emotions (Gratitude F1 0.813, Love F1 0.663, Anger F1 0.616).

The primary contributions of this thesis are: the design of a complete end-to-end NLP pipeline that runs entirely on-device; the implementation of quantized trans-

former architectures for mobile execution; an incremental clustering approach tailored for short personal entries; a privacy-centric application architecture without cloud components; an empirical evaluation of Matryoshka Representation Learning truncation on journal-domain clustering, showing that 256-dim Nomic embeddings retain over 99.5% of Spearman correlation relative to full 768-dim while losing 15 percentage points of V-measure against MiniLM on domain data; and the first benchmark of Apple NLEmbedding against dedicated sentence embedding models on semantic similarity and clustering tasks, conducted by bridging the Swift SDK into the Python evaluation pipeline via a standalone Swift script.

6.1 Future Work

Finer emotion taxonomy. The 12-class GoEmotions mapping collapses related emotions to make the classification task tractable on mobile hardware. A more granular taxonomy, or one calibrated specifically to journaling language, would improve the quality of emotion-based filtering and the cluster-naming heuristic.

Clustering stability. DBSCAN recomputes all clusters from scratch when a new entry arrives. For a corpus of several hundred entries this is acceptable, but it becomes slow and produces label churn as the user adds entries over weeks. Incremental DBSCAN or a hierarchical approach that reuses previous cluster assignments would reduce both latency and label instability.

Multimodal input. The current system processes only text. Short audio recordings or attached photos are common in mobile journaling workflows. Extending the embedding pipeline to fuse text with speech or image features would broaden the kinds of patterns the system can surface.

On-device MRL search. The MRL truncation flexibility studied here was applied only to clustering. The same property makes MRL embeddings attractive for fast approximate nearest-neighbour search: a user could search past entries using a 64-dimensional index for speed, then re-rank results using the full 256-dim vectors. This retrieval pattern would complement the existing clustering view without requiring a second model.

Schema migration and App Store readiness. The SwiftData schema currently wipes stored data on model changes. A versioned migration policy, a privacy policy URL, and a VoiceOver accessibility pass are the remaining steps before an App Store submission.